



Komunikasi Serial UART pada Sistem Embedded



Nama: Muhammad Fatih Nadif Pratama



NIM: 40040324650087



Program Studi: Teknologi Rekayasa Otomasi



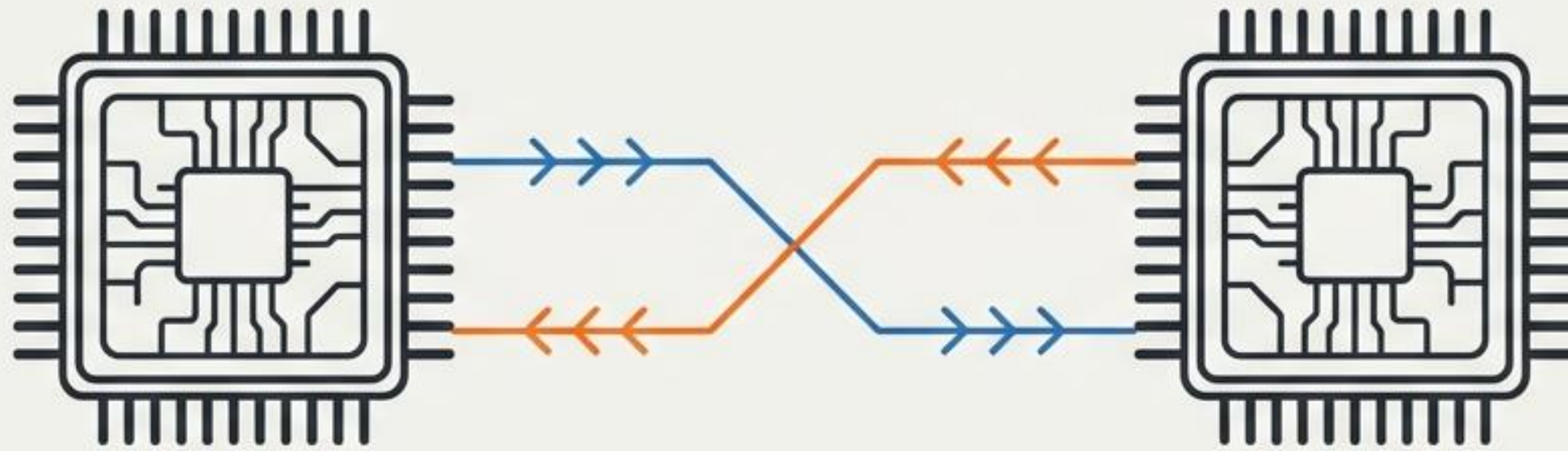
Fakultas: Sekolah Vokasi



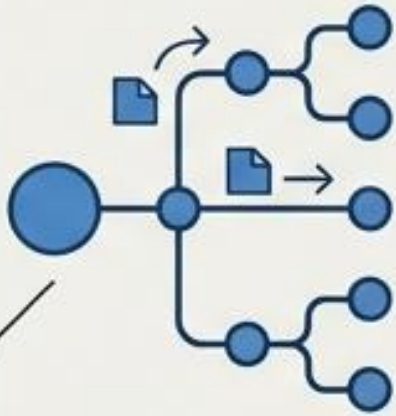
Universitas: Universitas Diponegoro

Komunikasi Serial UART pada Sistem Embedded

Arsitektur Transmisi Data, Struktur Frame, dan Implementasi pada STM32 & ESP32



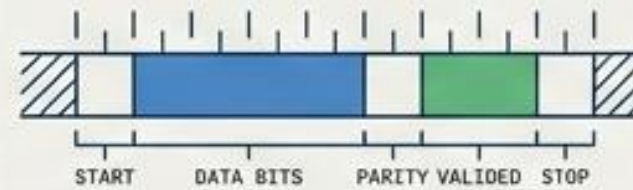
Tujuan Pembelajaran



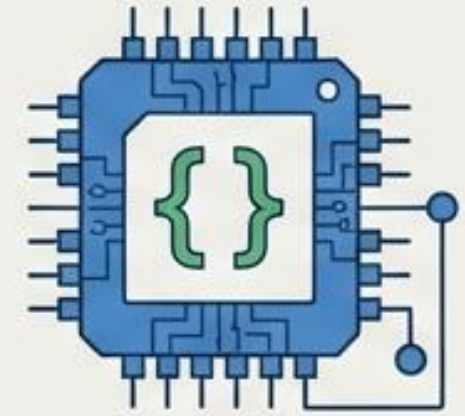
Memahami fundamental ekosistem pertukaran data antar perangkat.



Menguasai anatomi jalur fisik komunikasi UART.

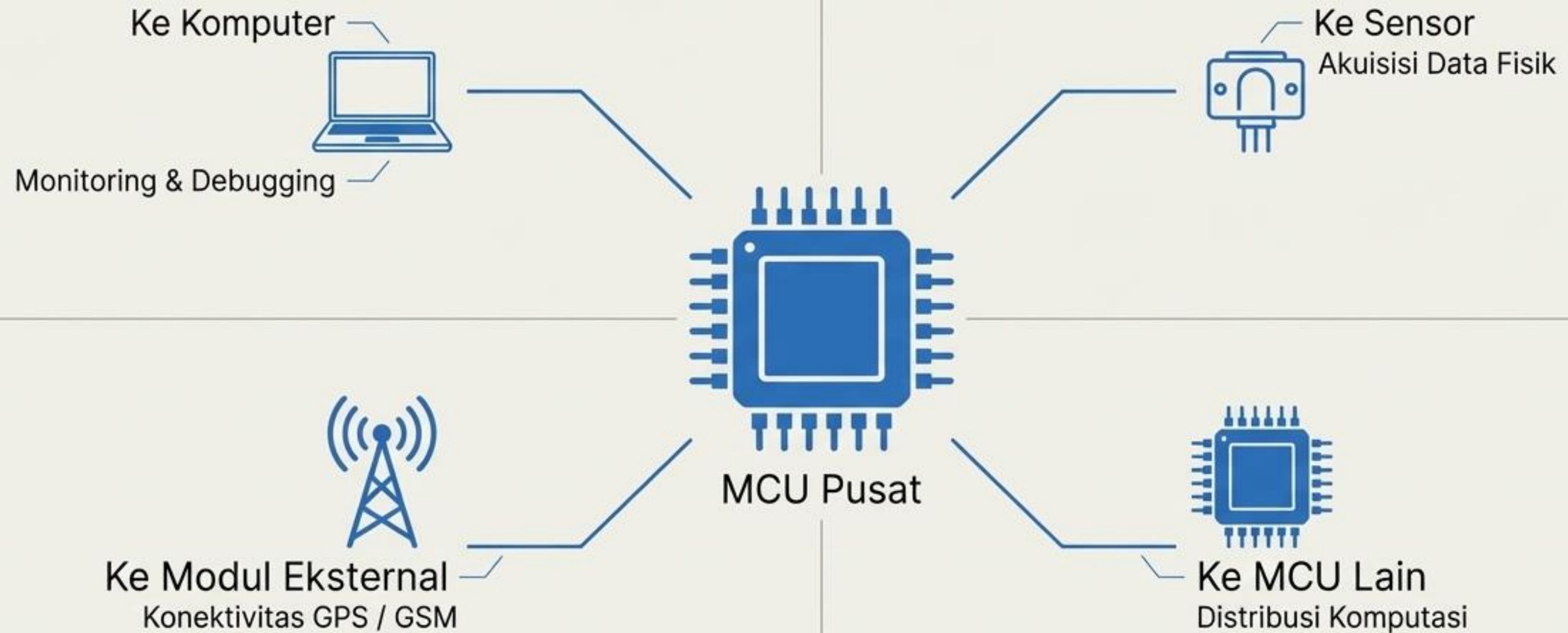


Menganalisis struktur frame dan parameter asinkron.



Mengimplementasikan transmisi data pada STM32 dan ESP32.

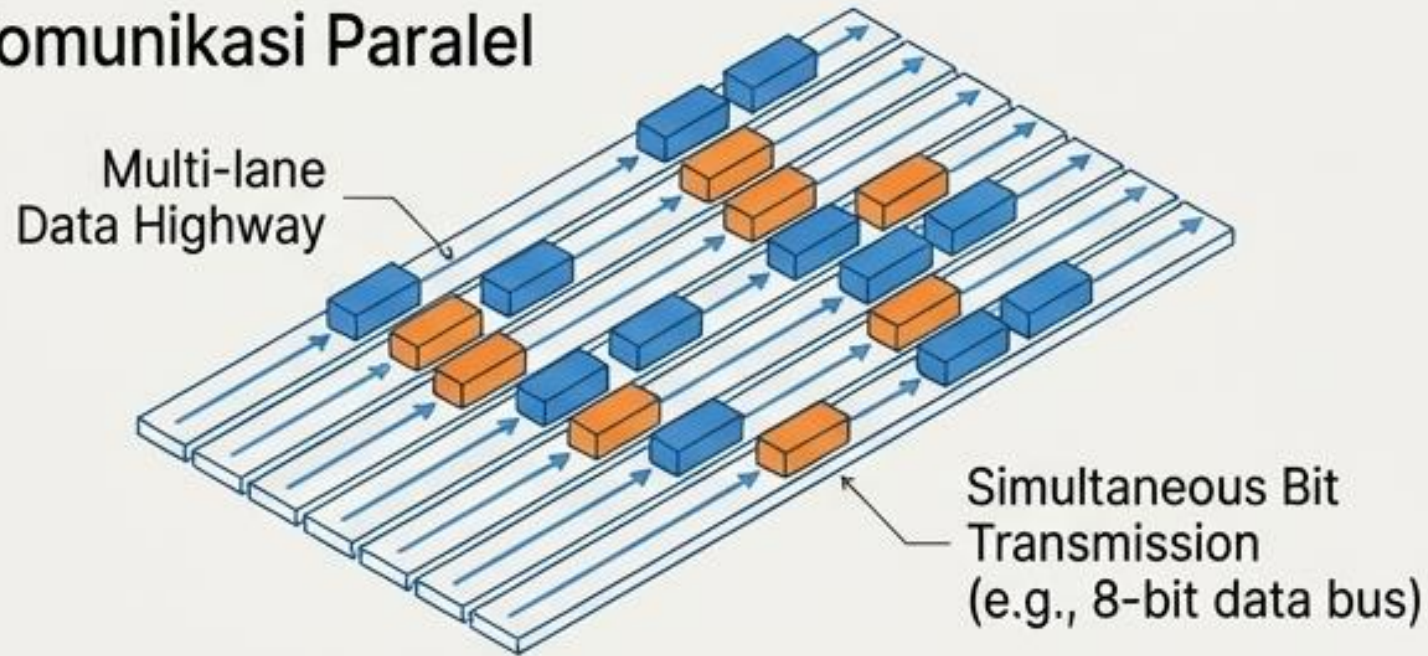
Pengantar Komunikasi Data pada Sistem Embedded



Sistem embedded tidak dapat bekerja dalam isolasi.
Pertukaran data mutlak membutuhkan protokol komunikasi yang disepakati.

Perbandingan Komunikasi: Serial vs Paralel

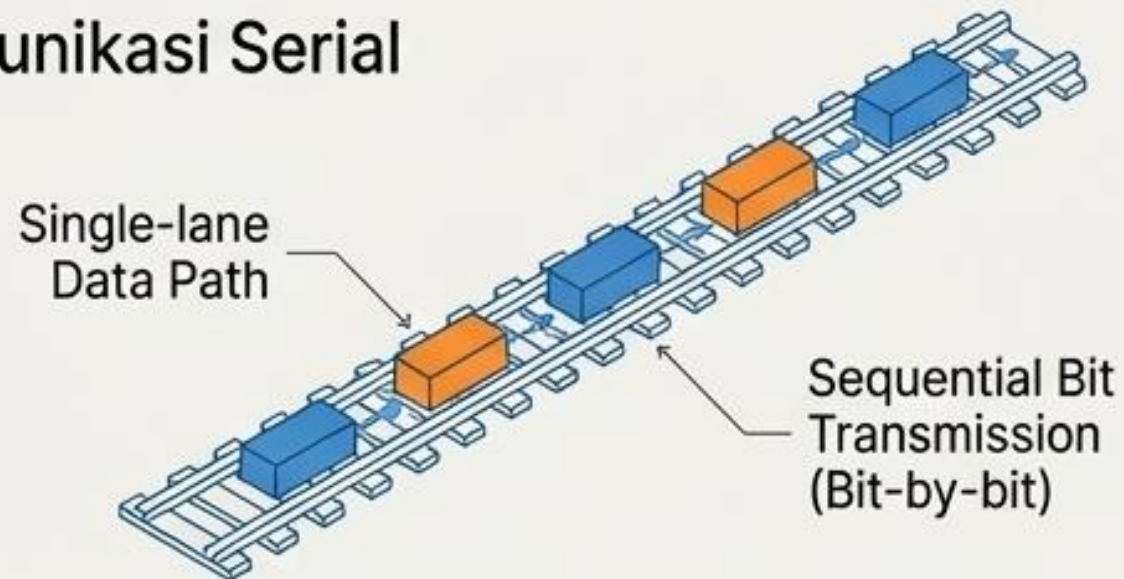
Komunikasi Paralel



Komunikasi Paralel

- Mengirim beberapa bit serentak (contoh: 8-bit data bus)
- Sangat cepat pada jarak pendek
- Butuh banyak kabel dan rentan interferensi sinyal

Komunikasi Serial

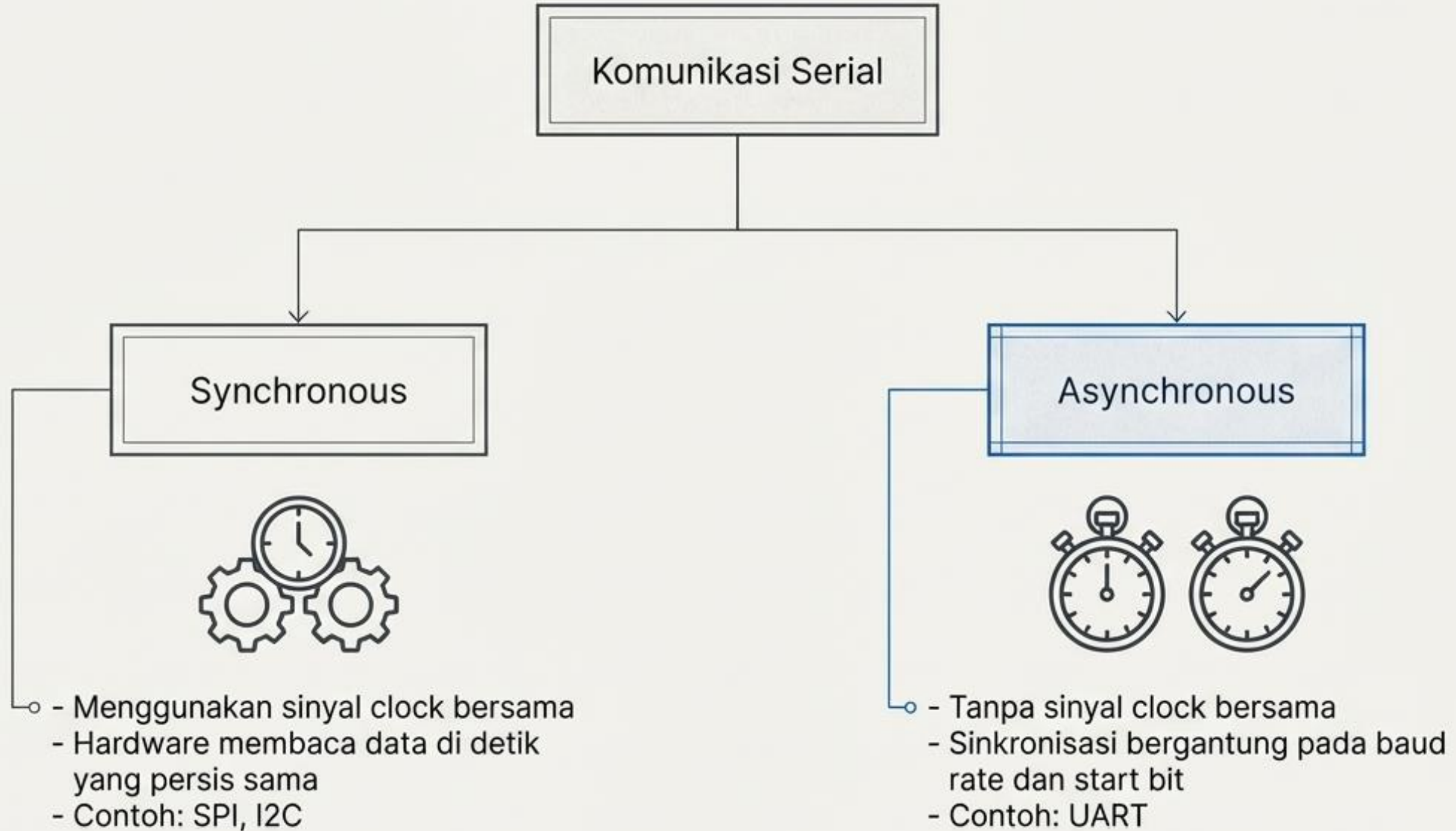


Komunikasi Serial

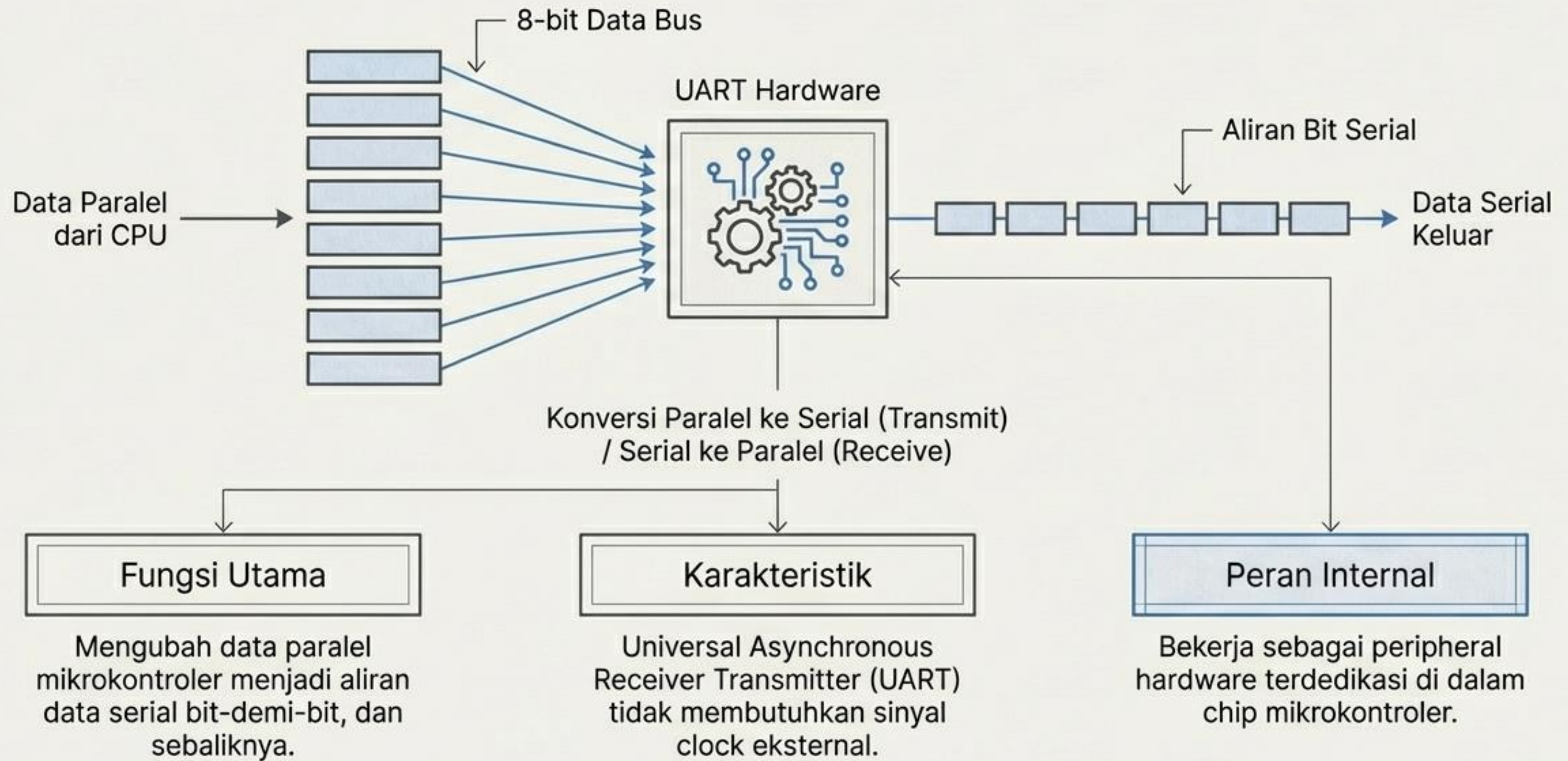
- Mengirim data bit-demi-bit berurutan melalui satu jalur
- Sangat stabil untuk pengiriman jarak jauh
- Desain kabel jauh lebih sederhana

Kesimpulan: Serial communication adalah standar de-facto pada sistem embedded modern.

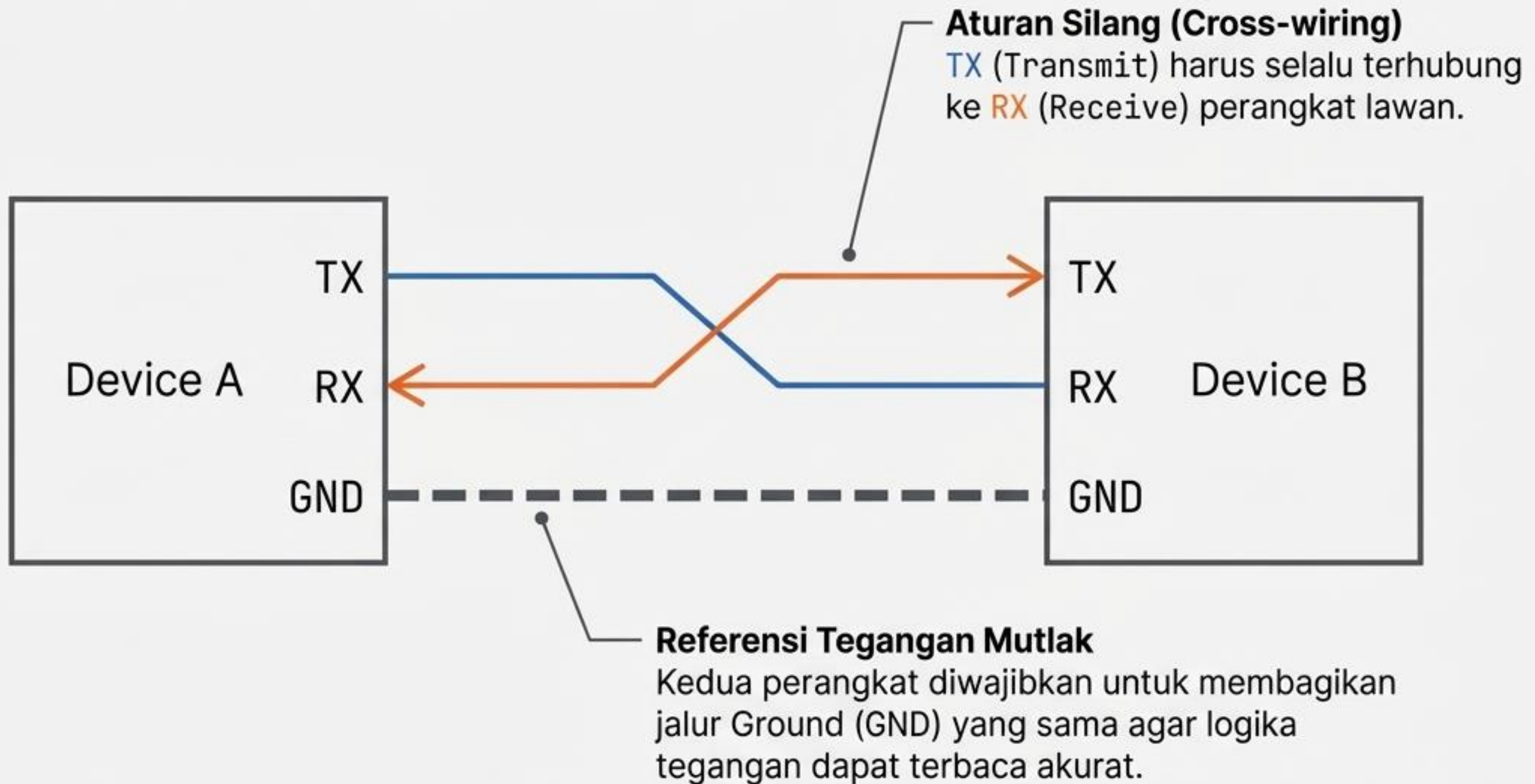
Taksonomi Protokol Komunikasi Serial



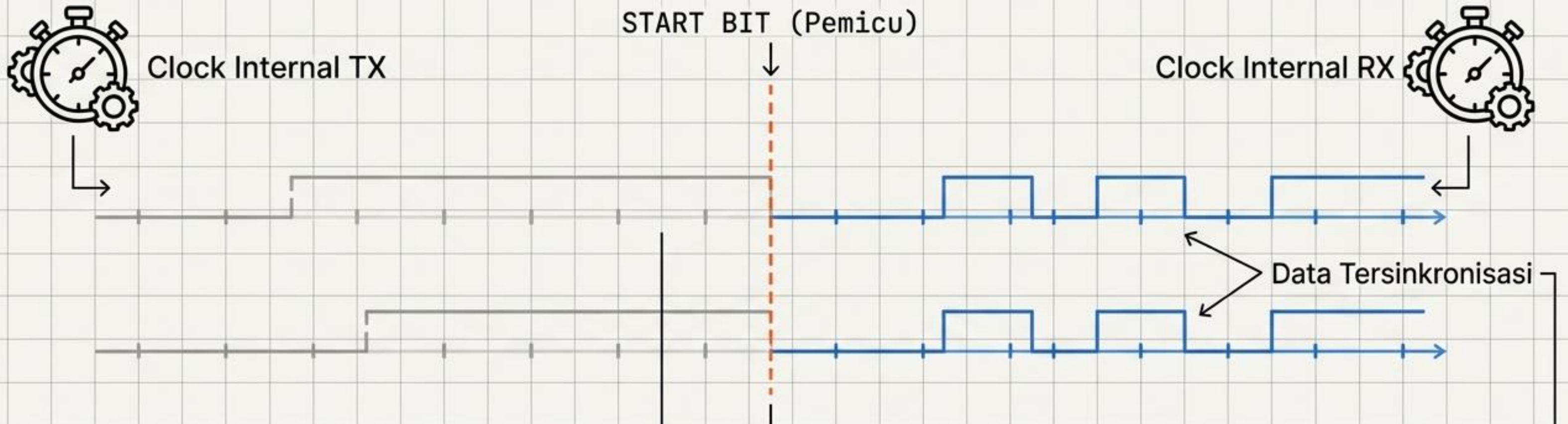
Definisi UART: Konversi Data Paralel ke Serial



Jalur Komunikasi Fisik UART



Mekanisme Sinkronisasi Asynchronous



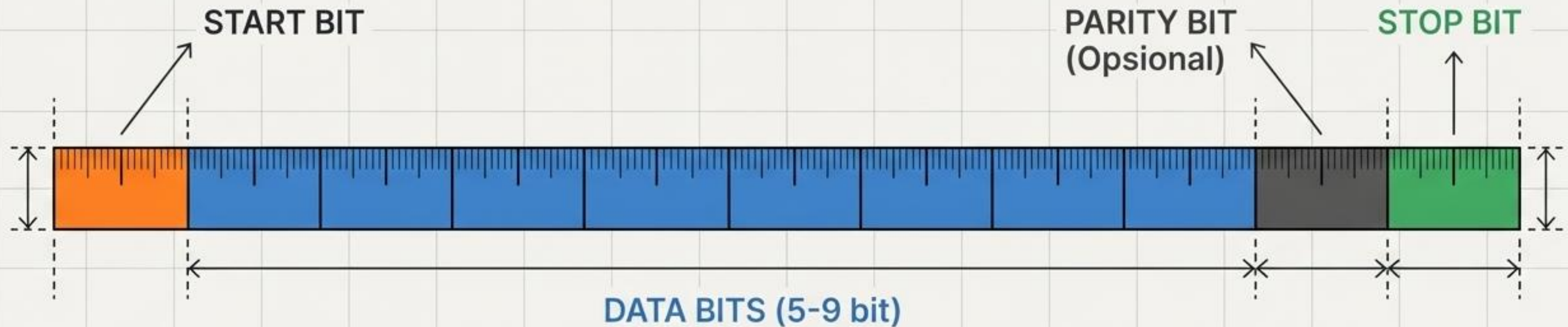
Tantangan Tanpa Clock Eksternal

Tanpa jalur clock fisik yang menghubungkan kedua perangkat, receiver tidak tahu kapan data akan mulai datang.

Solusi Asynchronous

1. Transmitter memicu sinyal transisi (Start Bit).
2. Receiver mendeteksi perubahan tegangan dan me-reset timer internalnya.
3. Keduanya lalu membaca bit demi bit berdasarkan kecepatan (Baud Rate) yang telah disepakati sebelumnya.

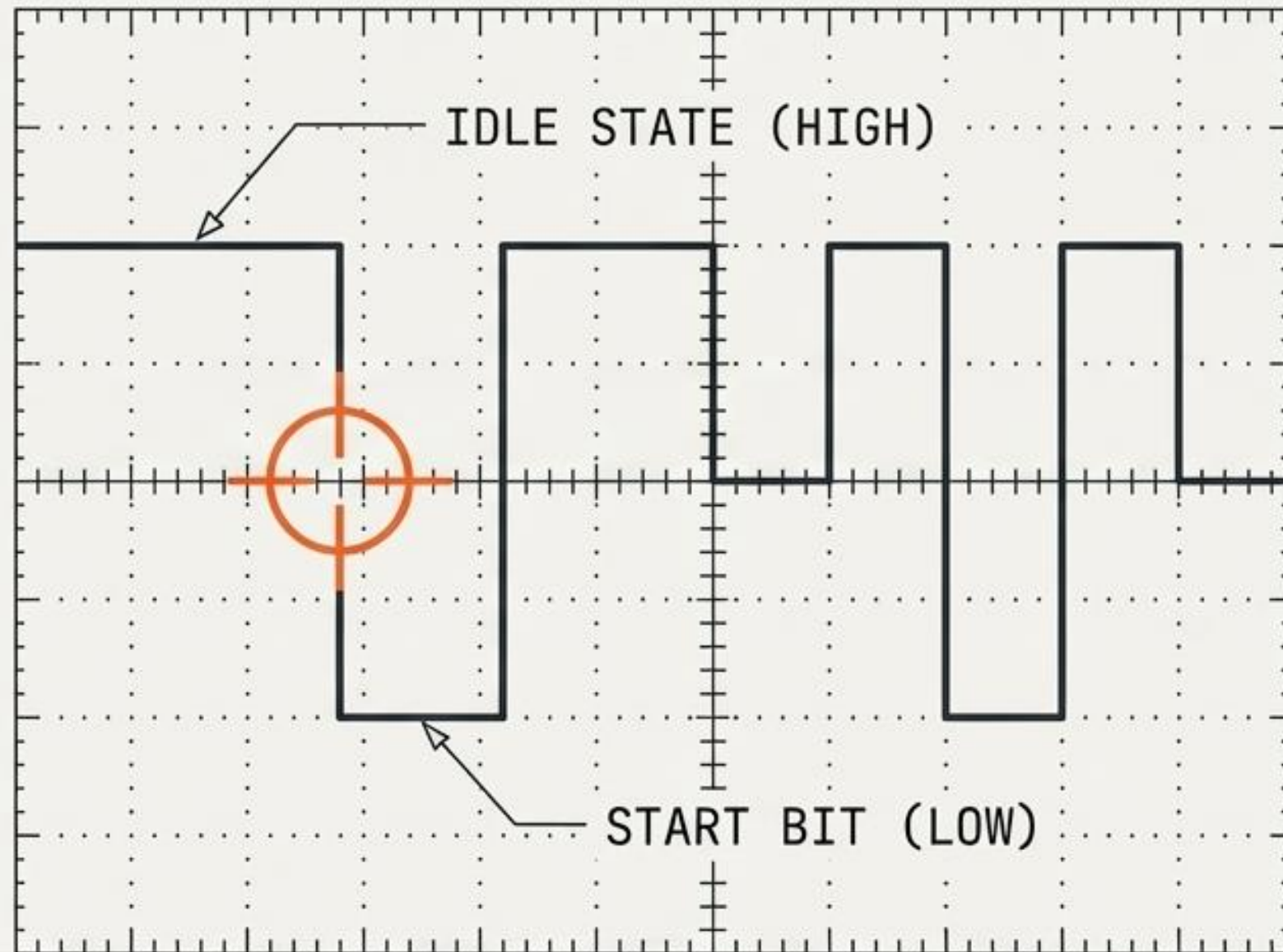
Anatomi Makro: Struktur Frame UART



Definisi Frame

Komunikasi UART membungkus setiap karakter data ke dalam satu paket (frame) utuh. Struktur ini mutlak diperlukan agar receiver dapat mengenali di mana sebuah karakter dimulai dan diakhiri di tengah aliran tegangan yang konstan.

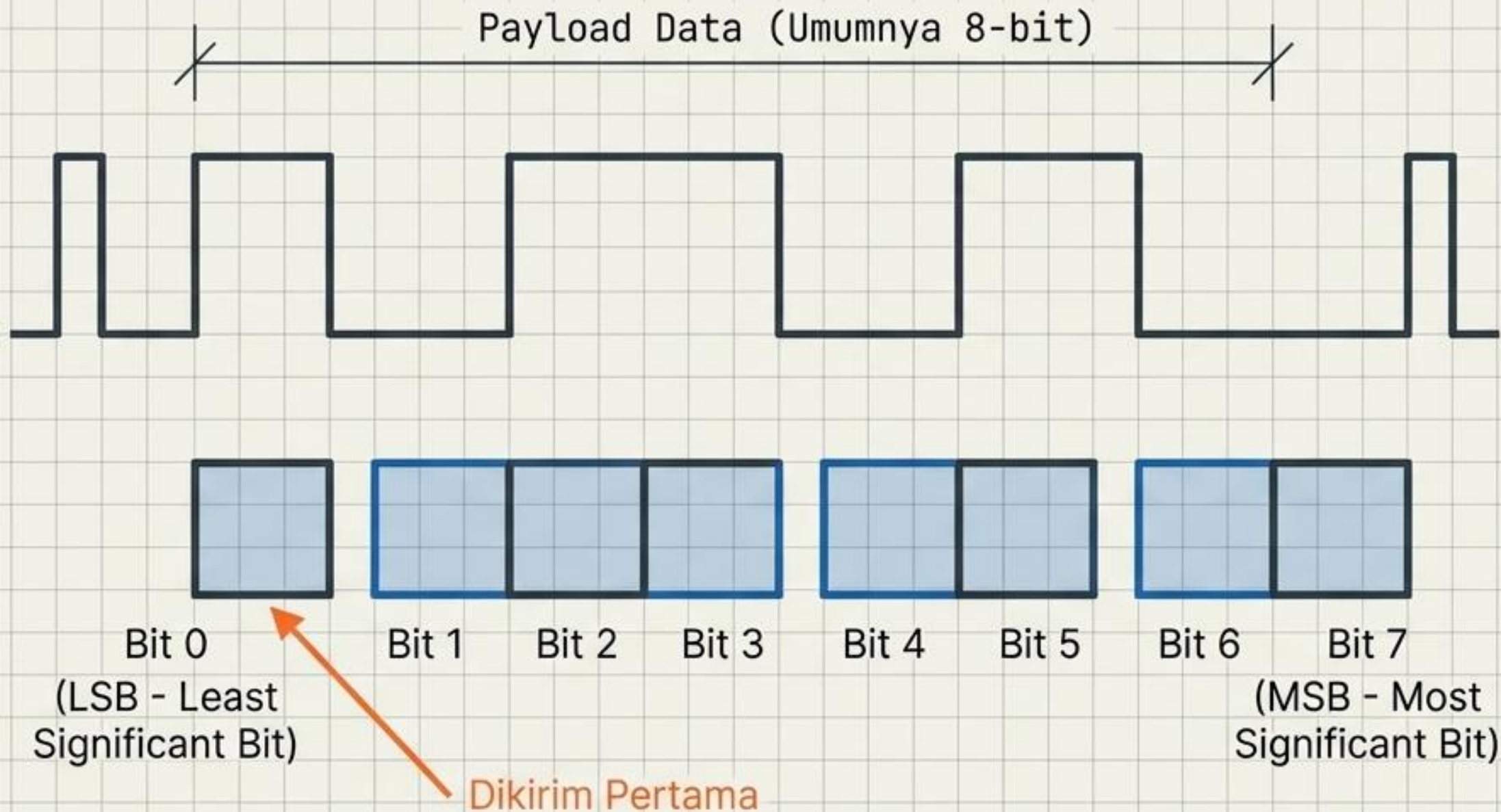
Fase 1: Start Bit (Sinyal Pemicu)



Membangunkan Receiver

- Kondisi normal (idle) jalur komunikasi selalu berada di logika HIGH.
- Start bit adalah transisi mendadak dari HIGH ke LOW.
- Penurunan tegangan ini bertindak sebagai interupsi yang memerintahkan receiver untuk mulai melakukan sampling data.

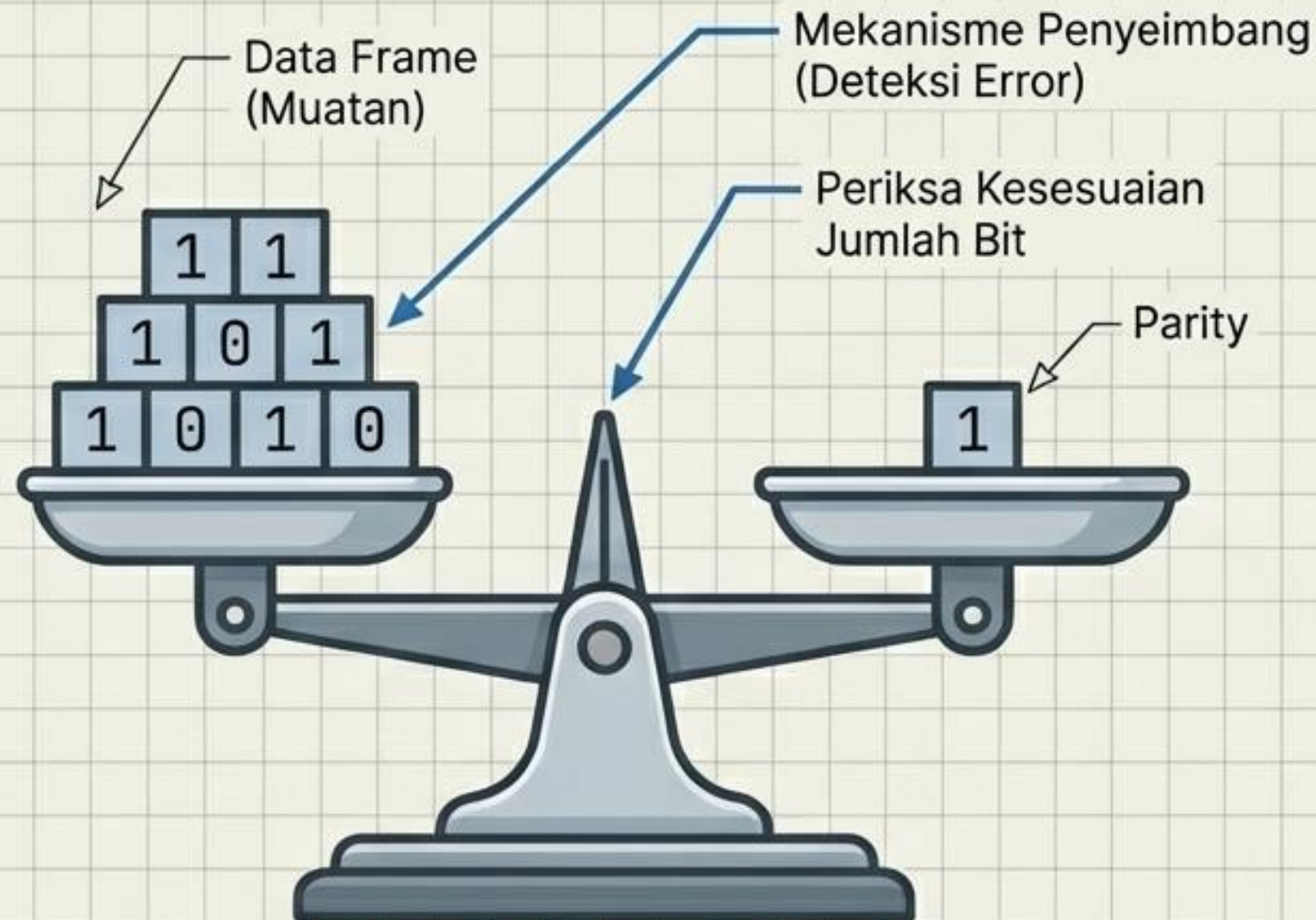
Fase 2: Data Bits (Muatan Utama)



Karakteristik Transmisi

- Lebar muatan data bisa dikonfigurasi antara 5 hingga 9 bit.
- Standar industri modern hampir selalu menggunakan konfigurasi 8-bit data.
- Data dibongkar dan dikirim dengan urutan LSB terlebih dahulu.

Fase 3: Parity Bit (Deteksi Error)



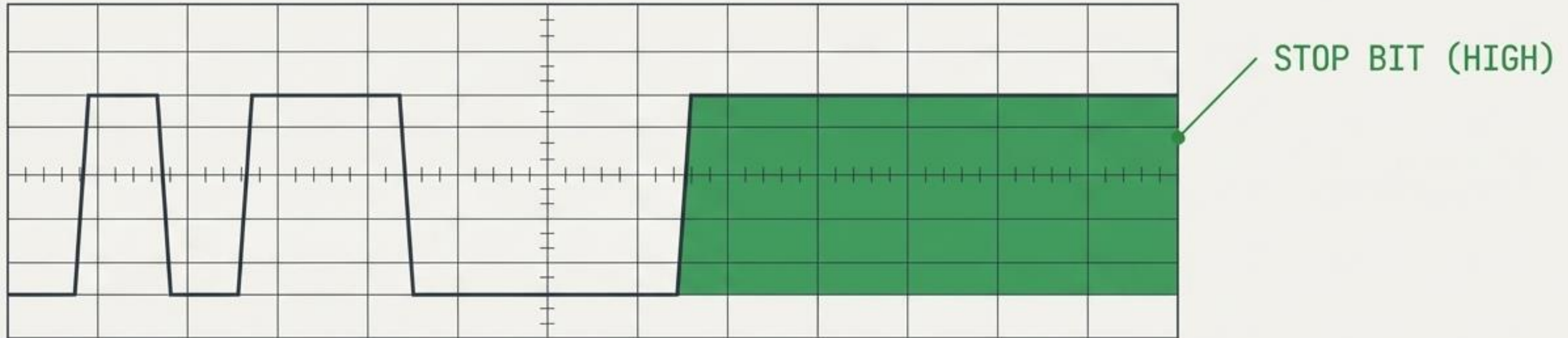
Parity Modes		
Even Parity (Genap)	Odd Parity (Ganjil)	None (Tanpa Parity)
- Mengatur bit parity agar total jumlah angka '1' dalam frame menjadi bilangan genap. 1 0 1 1 1 ✓	- Mengatur bit parity agar total jumlah angka '1' dalam frame menjadi bilangan ganjil. 1 0 1 1 0 ✓	- Sering digunakan untuk menghemat waktu transmisi jika error sangat jarang terjadi. 1 0 1 1 ⚠



Batas Kemampuan:

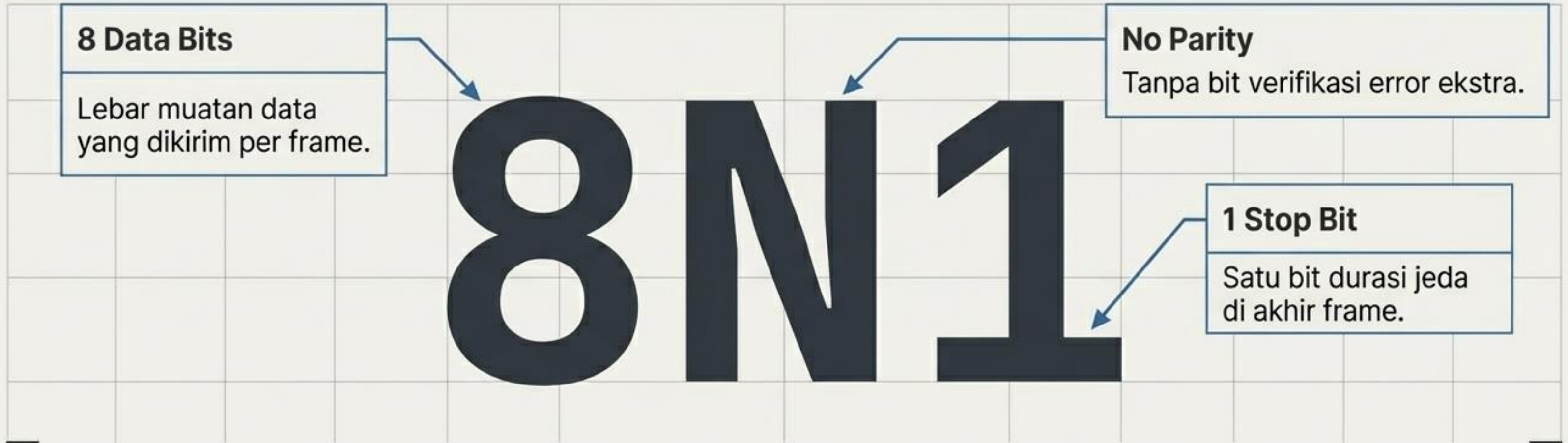
Parity bit hanyalah mekanisme deteksi error sederhana. Fitur ini dapat mengetahui adanya data korup, namun tidak dapat memperbaiki (koreksi) error tersebut.

Fase 4: Stop Bit (Jeda Eksekusi)



- - **Identifikasi Akhir:** Sinyal ini secara absolut menginformasikan receiver bahwa satu frame data telah selesai dikirim. (JetBrains Mono)
- - **Kondisi Logika:** Selalu direpresentasikan dengan tegangan HIGH. (JetBrains Mono)
- - **Waktu Pemrosesan:** Memberikan jeda krusial bagi mikrokontroler untuk memindahkan data dari buffer ke memori utama sebelum frame berikutnya menabrak masuk.
- - **Konfigurasi:** Umumnya diatur sebanyak 1 bit atau 2 bit. (JetBrains Mono)

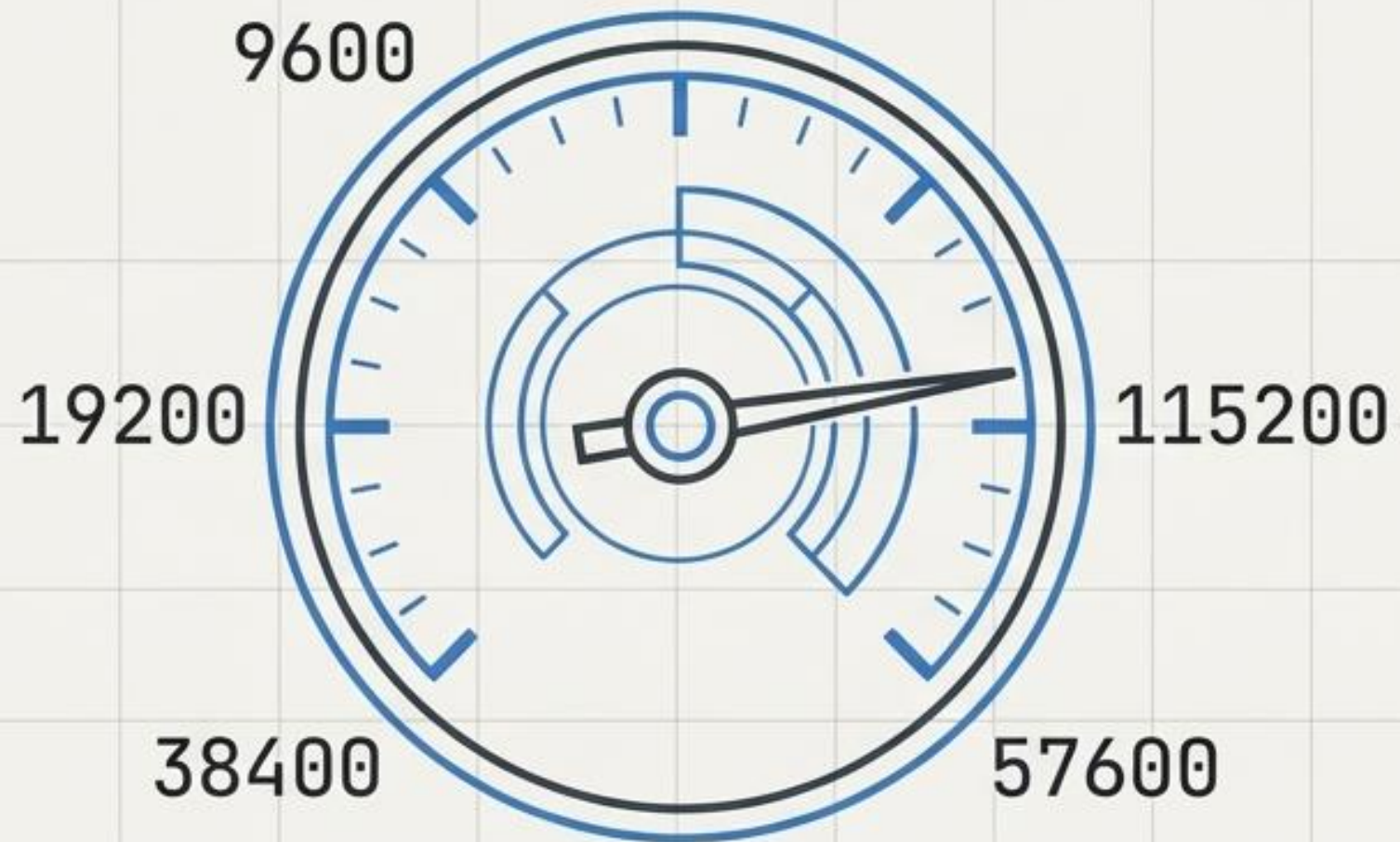
Sintesis Standar Format: 8N1



$1 \text{ Start Bit} + 8 \text{ Data Bits} + 0 \text{ Parity} + 1 \text{ Stop Bit} = 10 \text{ Bit Total}$

8N1 adalah format konfigurasi serial paling universal di industri. Untuk mengirim 1 byte karakter (**8-bit**), sistem secara fisik mengirimkan 10 bit data melalui jalur kabel.

Denyut Nadi Sistem: Baud Rate



Definisi Baud Rate

Jumlah bit absolut yang ditransmisikan per detik melalui jalur komunikasi.

Aturan Kalibrasi Mutlak

Kedua perangkat (TX dan RX) wajib dikonfigurasi pada angka baud rate yang persis sama. Tanpa kesepakatan ini, receiver akan membaca data pada interval yang salah.



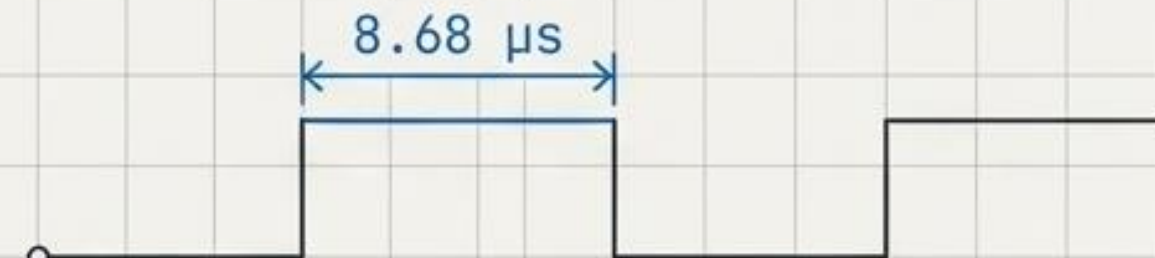
Jika Baud Rate tidak cocok, sistem akan mengalami Framing Error massal dan data yang diterima hanya berupa karakter acak (gibberish).

Matematika Transmisi: Kalkulasi Bit Time

$$\text{Bit Time} = \frac{1}{\text{Baud Rate}}$$

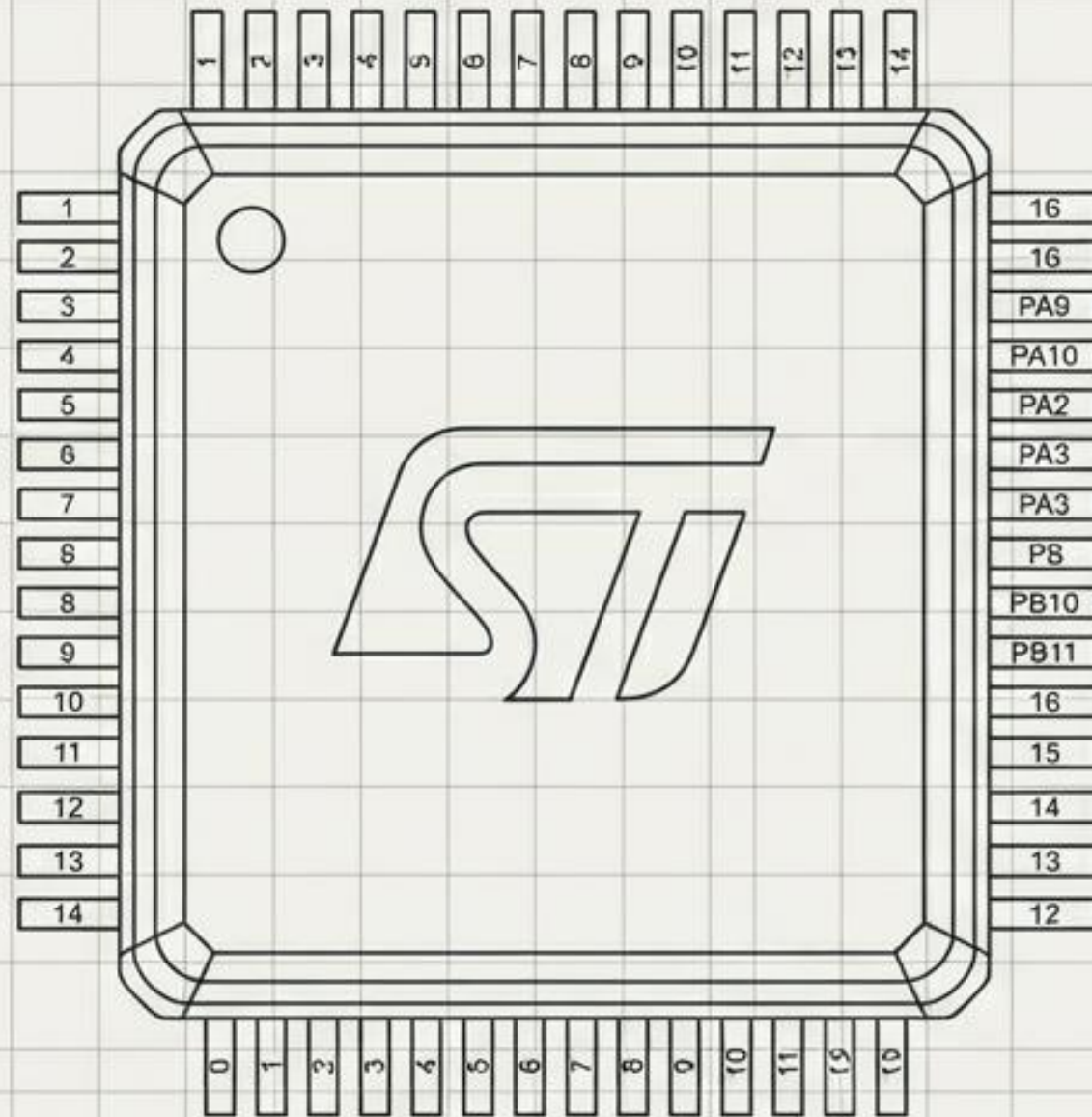
Contoh Eksekusi Hardware:

- Baud Rate ditetapkan pada: 115200 bps
- Kalkulasi: $1 / 115200$
- Hasil: ≈ 8.68 mikrodetik (μs)



Bit Time adalah durasi presisi yang dibutuhkan mikrokontroler untuk menahan voltase sinyal pada jalur agar dapat dibaca sebagai satu bit yang valid oleh receiver.

Spesifikasi Hardware: UART pada STM32



Mapping Pin Default (Hardware USART)

- USART1 -> TX: PA9 | RX: PA10
- USART2 -> TX: PA2 | RX: PA3
- USART3 -> TX: PB10 | RX: PB11

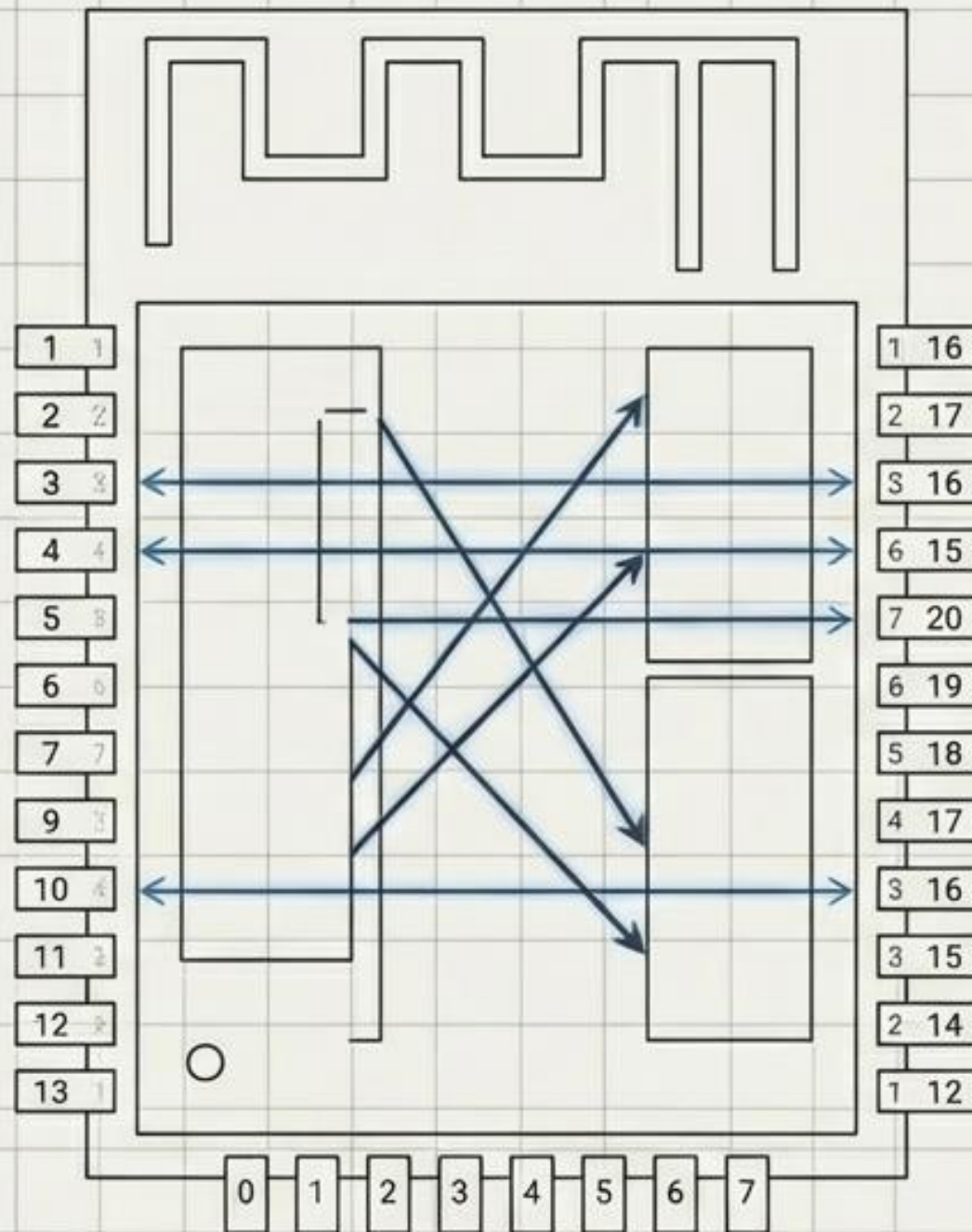


Fitur Enterprise STM32

Peripheral USART pada arsitektur ARM Cortex-M STM32 dilengkapi dengan dukungan tingkat lanjut untuk beban kerja industri:

- Interrupt-driven reception
- Direct Memory Access (DMA) untuk zero-CPU overhead
- Hardware Flow Control (RTS/CTS)

Spesifikasi Hardware: UART pada ESP32



Infrastruktur Peripheral

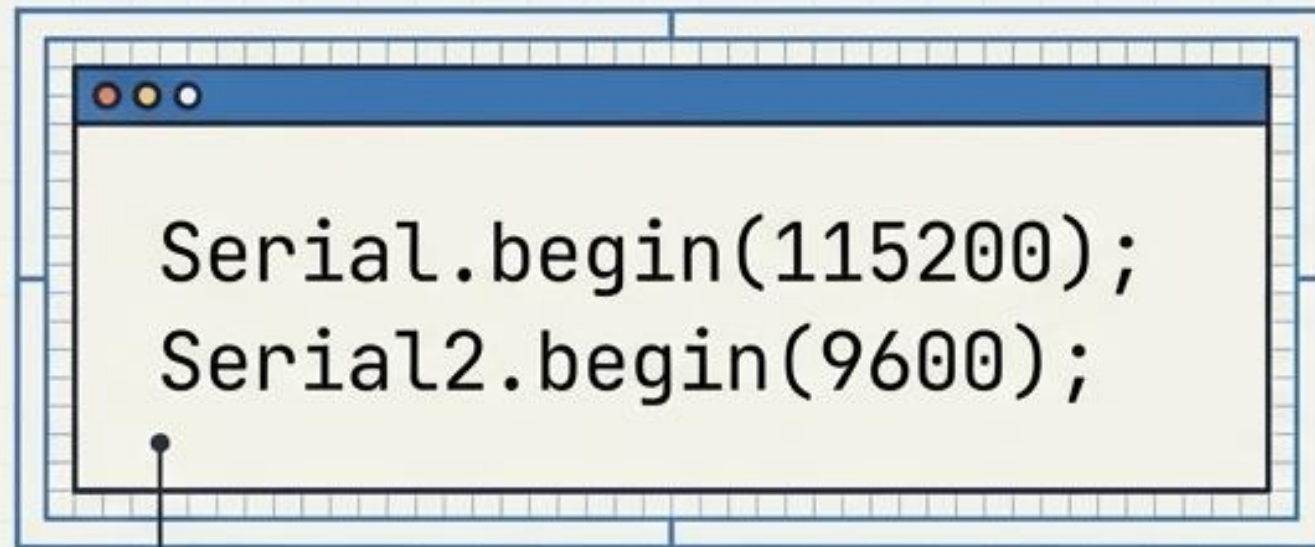
- ESP32 memiliki 3 unit hardware terdedikasi: UART0, UART1, dan UART2.
- UART0 secara default dikunci untuk komunikasi USB/Flashing program ke komputer.



Keunggulan Arsitektur: GPIO Matrix

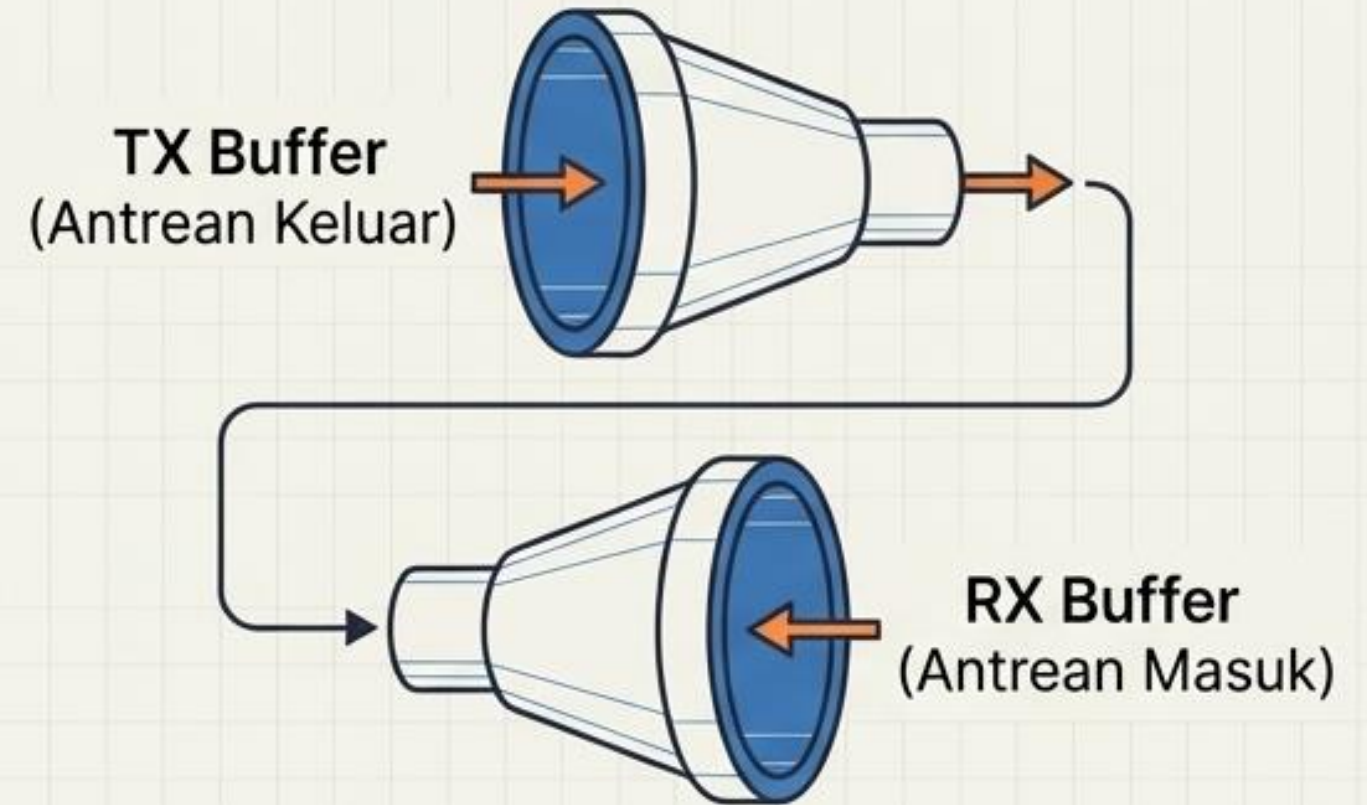
Berbeda dengan STM32 yang memiliki pin statis, ESP32 memiliki fitur internal GPIO Matrix. Ini memungkinkan engineer untuk memetakan (routing) jalur TX dan RX ke hampir semua pin digital yang tersedia secara fleksibel melalui perangkat lunak.

Inisialisasi Firmware & Manajemen Buffer



```
Serial.begin(115200);  
Serial2.begin(9600);
```

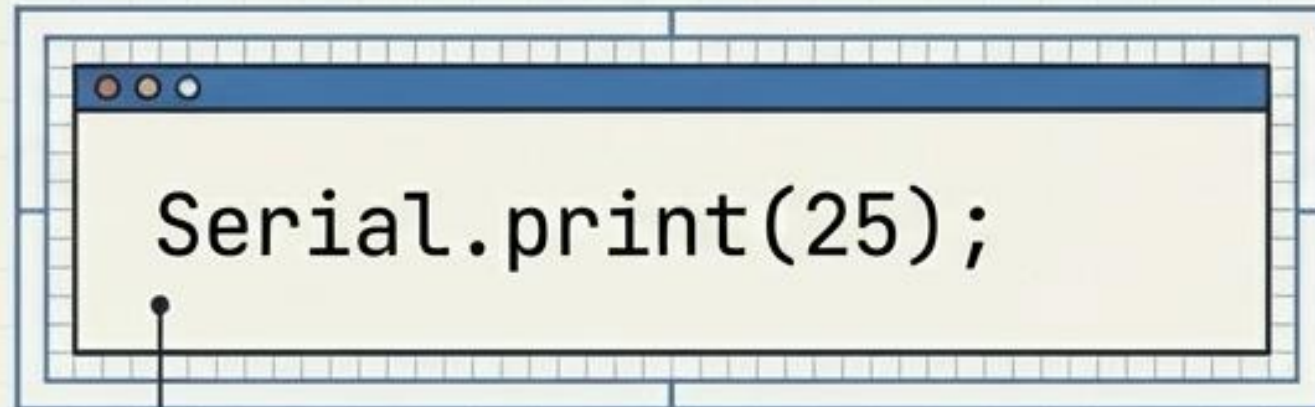
Perintah di atas (Arduino Framework) memerintahkan hardware untuk membuka jalur serial dan mengunci baud rate agar sinkron dengan receiver.



UART bekerja di latar belakang. Saat data tiba, hardware menyimpannya di RX Buffer sementara. Jika CPU tidak membaca memori ini dengan cukup cepat dan buffer penuh, maka akan terjadi Buffer Overflow.

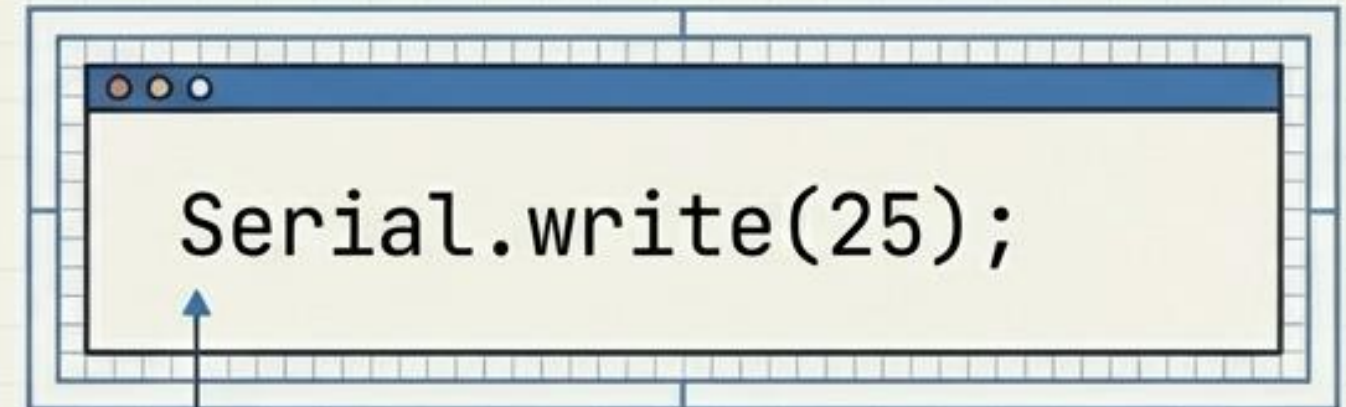
Siklus Instruksi: Mengirim Data (Transmit)

Transmisi Teks (Human-Readable)



Mengubah angka secara otomatis menjadi teks ASCII sebelum dikirim. Cocok untuk debugging dan dibaca oleh manusia melalui Serial Monitor.

Transmisi Biner (Raw Data)



Mengirim nilai byte biner murni secara langsung ke jalur kabel. Jauh lebih efisien, memakan sedikit bandwidth, dirancang untuk komunikasi mesin-ke-mesin.

*KESIMPULAN
dan PENUTUP*